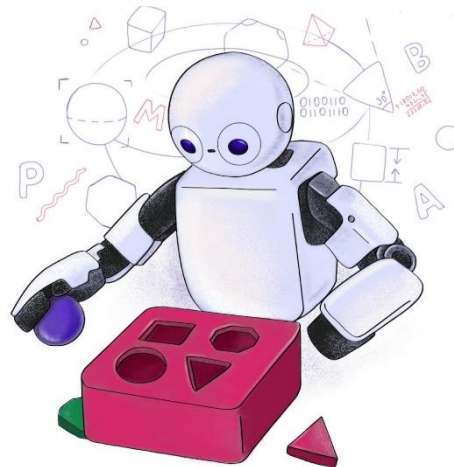


C24 - Inteligência Artificial: *k-Means*



Recapitulando

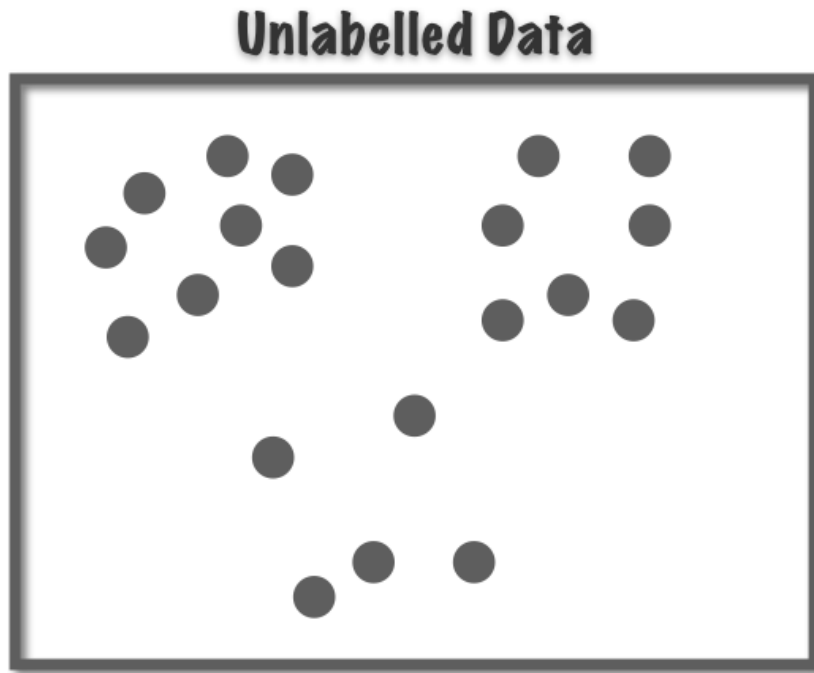
- Até o momento, todos os algoritmos que aprendemos seguiam o paradigma do aprendizado supervisionado.
- Hoje, falaremos sobre ***clustering***, que são algoritmos de **aprendizado não-supervisionado**.
 - Não temos rótulos, apenas atributos.
- Eles visam criar agrupamentos de dados (chamados de ***clusters*** ou **grupos**) segundo seu **grau de semelhança**.
- Nesta aula, aprenderemos sobre o algoritmo chamado de **k-Médias** (ou ***k-Means***, em inglês) que é um dos algoritmos mais simples de ***clustering***.

clustering be like:



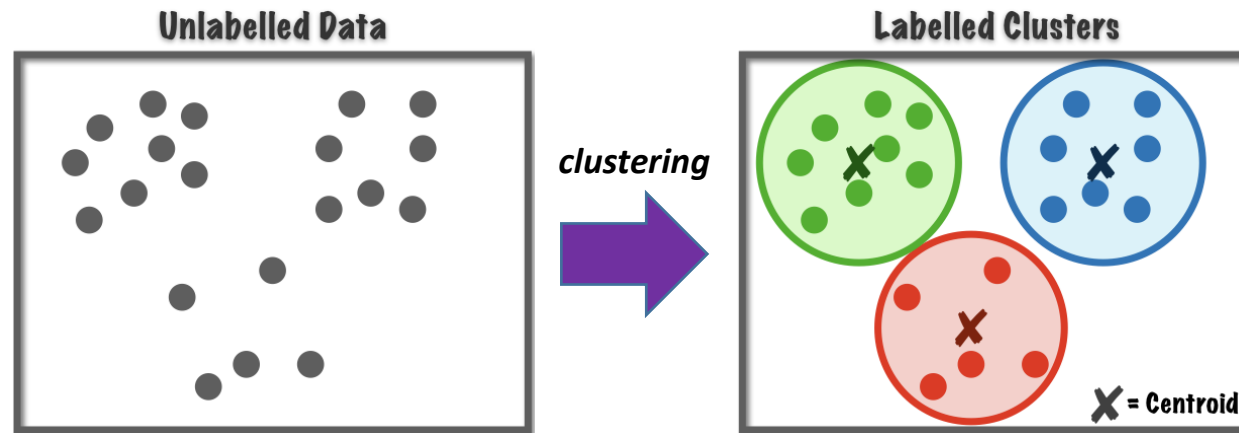
A **clusterização** organiza dados como na figura: cada ponto encontra seu grupo mais próximo, formando *clusters*.

Motivação



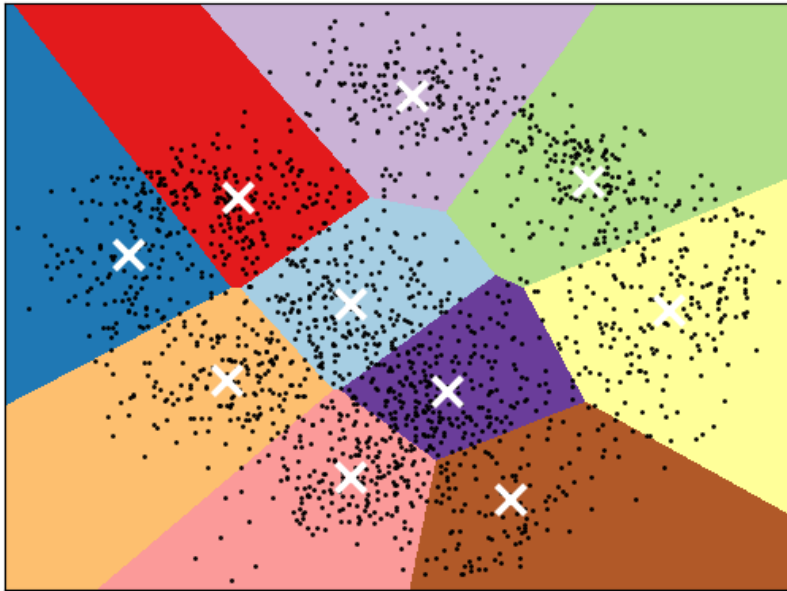
- O que podemos fazer se não tivermos informações sobre as classes (i.e., rótulos) a que pertencem os exemplos de entrada?

Motivação



- Veremos que informações úteis podem ser obtidas mesmo de exemplos cujas classes não são conhecidas.
- Enquanto o **aprendizado supervisionado** se concentra na **indução de classificadores**, o **aprendizado não-supervisionado** está interessado em **extrair estrutura e conhecimento de dados quando não existem rótulos**.
- *Clustering* tenta reproduzir como humanos organizam o mundo: agrupamos objetos por similaridade

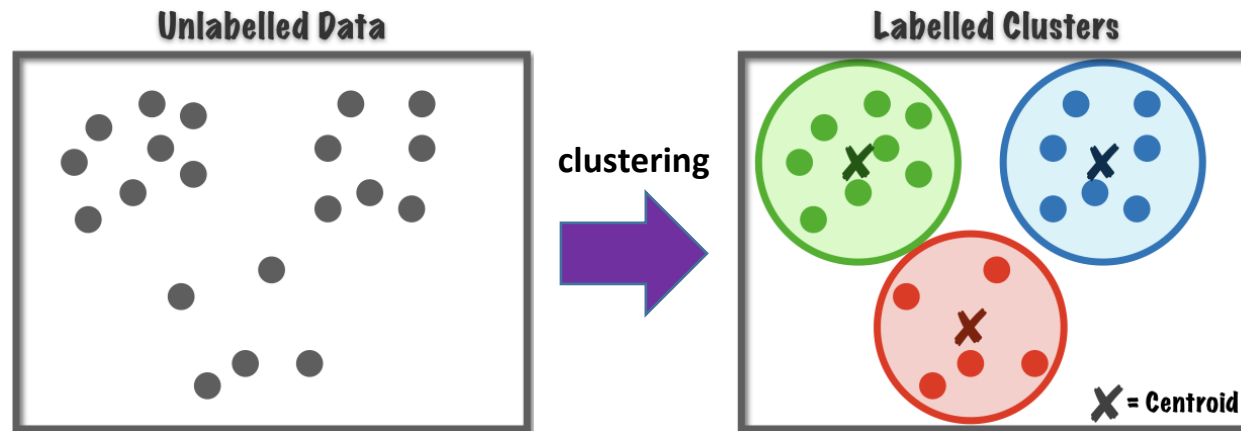
Motivação



- A tarefa mais popular dos algoritmos deste paradigma é a procura por **grupos** (chamados *clusters*) de **exemplos semelhantes**.
- O *clustering* agrupa dados **com base em similaridade** sem precisar de supervisão.
- A ideia mais importante é revelar a **estrutura intrínseca dos dados**:
 - padrões escondidos
 - subgrupos naturais
 - relações não explícitas

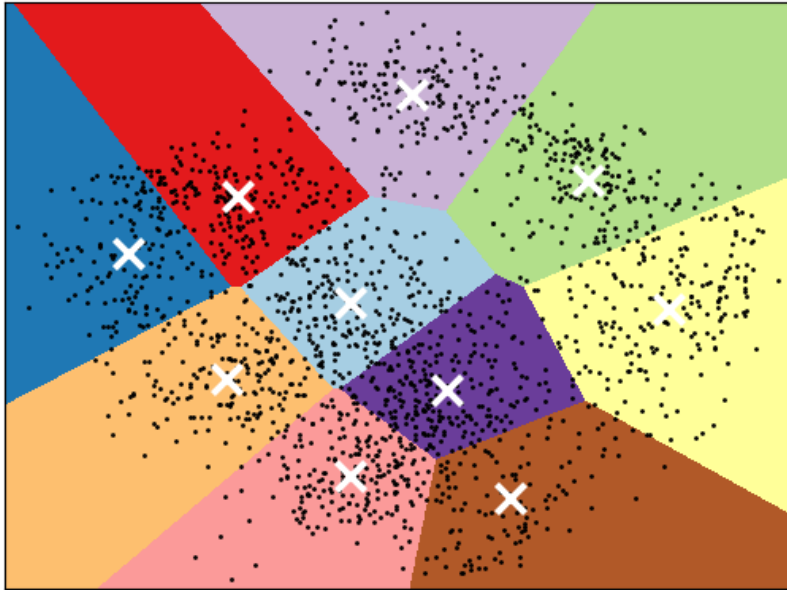
Identificação de *clusters*

- A **tarefa fundamental** do **aprendizado não-supervisionado** é a **identificação de *clusters***.
- Nessa tarefa, a **entrada** é um conjunto de **vetores de atributo** (i.e., exemplos), **mas sem rótulos**.
- A **saída** é um conjunto de ***clusters*** a que pertencem cada um dos exemplos do conjunto de treinamento.



OBS.: A identificação visual de clusters em um espaço bidimensional é fácil, mas em quatro ou mais dimensões isso já não é mais possível. Nesses casos, apenas algoritmos de identificação de clusters conseguem agrupar os dados.

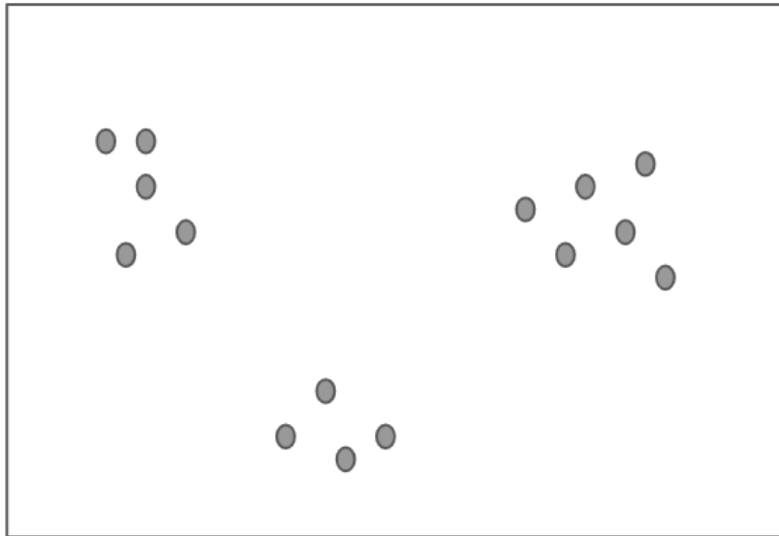
Aplicações



- Redução de complexidade
 - substituir milhões de pontos por alguns grupos
 - resumir dados complexos
- Pré-processamento para outros modelos
 - criar features (cluster ID)
 - inicializar modelos
 - reduzir dimensionalidade
 - organizar dados antes de aprendizado supervisionado
- Detecção de padrões raros
 - detector de comportamento “normal”
 - tudo fora disso é considerado suspeito

Como representar os *clusters*?

Unlabelled Data



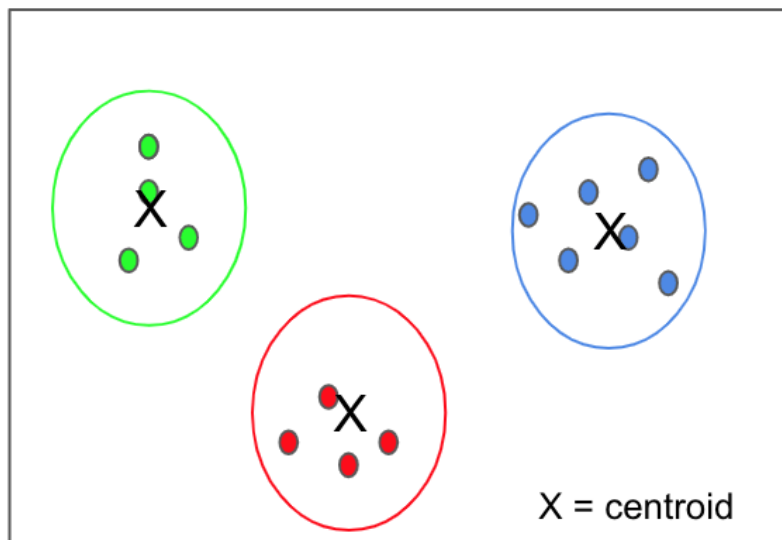
- Para realizar a identificação, temos que **decidir como os *clusters* serão representados**.
- Existem algumas opções como a densidade, distribuição, hierarquia, etc.
- Porém, a abordagem mais simples usa os **centroides** (i.e., centros) dos clusters.
- O algoritmo conhecido como ***k-Means*** implementa esta abordagem.
- Vejamos então suas características e como funciona.

Centroide

- **Centroide** é o ponto central de um *cluster* de dados.
 - Representa a "média" das amostras dentro do cluster.
- Os **centroides** podem ser usados como
 - centros para inicializar redes neurais Bayesianas ou de funções de base radial (RBF),
 - estimativas de valores de atributos ausentes.
 - forma de comprimir dados usando apenas o centróide mais próximo.
 - forma de visualizar dados multidimensionais.
 - *feature engineering*: o índice do centróide e a distância até os centróides podem ser novos atributos.
 - forma de detectar anomalias: distância alta ao centróide indica *outlier*.
 - forma de agrupar dados sem classes em problemas semi-supervisionados: os *clusters* são usados como classes e, conseqüentemente, rotular os dados.

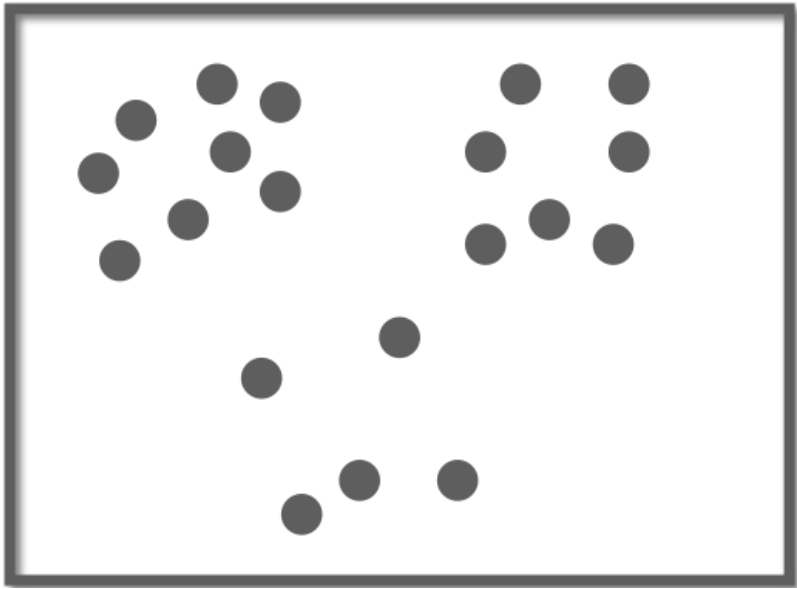
Como representar os *clusters*?

Labelled Data



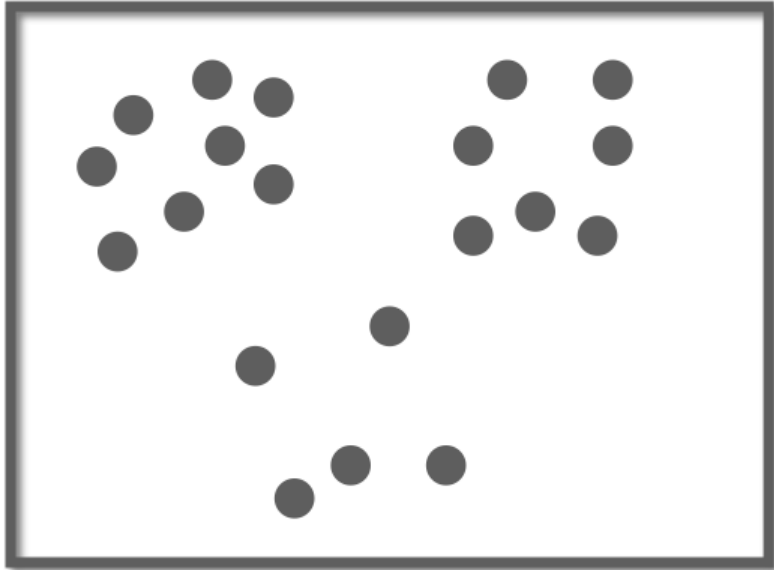
- Por exemplo, suponhamos os seguintes **vetores de atributos** em um espaço bidimensional, x_1 e x_2 : (2, 5), (1, 4), (3, 6).
- Nesse caso, se elas pertencem a um *cluster*, seu **centroide** é representado pelo vetor (2, 5), pois
 - A média do primeiro atributo é $\frac{2+1+3}{3} = 2$.
 - A média do segundo atributo é $\frac{5+4+6}{3} = 5$.
- **OBS.:** Em geral, aplicamos clusterização apenas a atributos numéricos.
 - Existem algoritmos de clusterização específicos para dados categóricos: *k-modes* e *k-prototypes*.

Quantos devem ser os *clusters*?



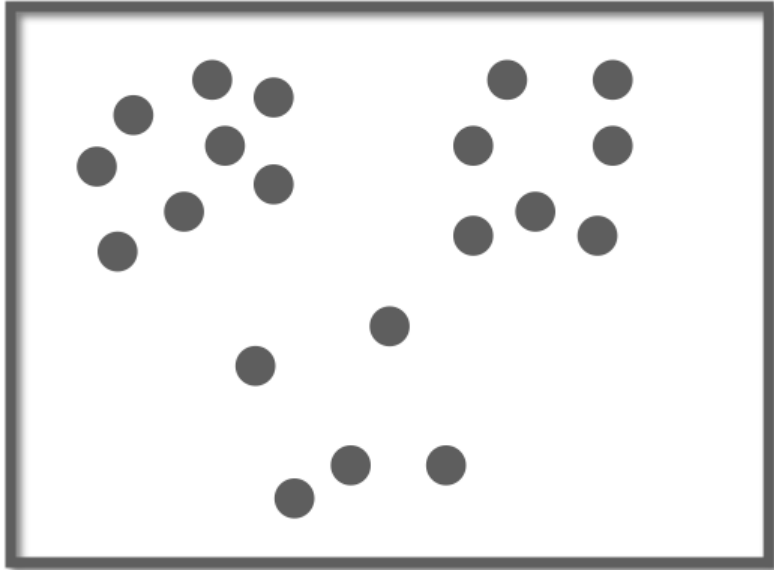
- Os *clusters* não devem se sobrepor.
- Cada amostra deve **pertencer** a **um e apenas um *cluster***.
- Porém, **dentro do mesmo cluster**, as **amostras** devem estar relativamente **próximas** umas das outras e **distantes** das **amostras dos outros *clusters***.
- Aí surge uma dúvida.
- Quantos *clusters* um conjunto de dados contém?

Quantos devem ser os *clusters*?



- Na figura conseguimos identificar visualmente três clusters.
- No entanto, o número de opções existentes não se limita a essa única possibilidade:
 - em um extremo, todo o conjunto de dados pode ser pensado como formando um grande cluster;
 - no outro, cada exemplo pode ser visto como representando seu próprio cluster de um único exemplo.

Quantos devem ser os *clusters*?



- **Implementações** práticas geralmente evitam esse problema pedindo ao **usuário que forneça o número de *clusters***.
- Existem diversos métodos para otimizar o hiperparâmetro que define o número de *clusters*, dentre eles, o mais usado é o método do cotovelo.
- Veremos como ele funciona em breve.
- Porém, antes, precisamos entender como os *clusters* são criados.

Medindo distâncias

- Algoritmos para identificação de *clusters* precisam de uma **métrica para avaliar a distância entre uma amostra e o centroide** de um *cluster*.
- Uma forma de fazer isso é usar a **distância euclidiana** entre os dois vetores

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^K (x_i - y_i)^2},$$

onde K é o número de atributos e x_i e y_i são os i -ésimos elementos dos vetores tendo sua distância calculada.

A qual *cluster* um exemplo deve pertencer?

- Vamos supor que existam N clusters cujos centroides são denotados pelo vetor, $\mathbf{c}_i, \forall i \in (1, N)$.
- Uma amostra $\mathbf{x}$ tem uma certa distância $d(\mathbf{x}, \mathbf{c}_i)$ para cada centroide.
- Se $d(\mathbf{x}, \mathbf{c}_p)$ é a menor dessas distâncias, então, é natural colocarmos $\mathbf{x}$ como pertencente ao centroide $\mathbf{c}_p$, ou seja, o p -ésimo cluster.
- Portanto, escolhemos a **menor distância** para **definir a qual cluster um exemplo pertence**.
- Agora, veremos como funciona o algoritmo *k-Means*.

k-Means

- O "*k*" é um hiperparâmetro que define o número de *clusters*.
- O pseudocódigo do algoritmo é mostrado abaixo.

Entradas: conjunto de exemplos e número de *clusters*, *k*.

1. Defina *k* centroides iniciais (geralmente, feito de forma aleatória).

2. Repita

- a) Calcule a distância de cada exemplo, *x*, para cada um dos *k* centroides e atribua cada exemplo ao *cluster* mais próximo.
- b) Calcule o novo centroide de cada *cluster**

3. Enquanto as posições dos centroides continuarem mudando.

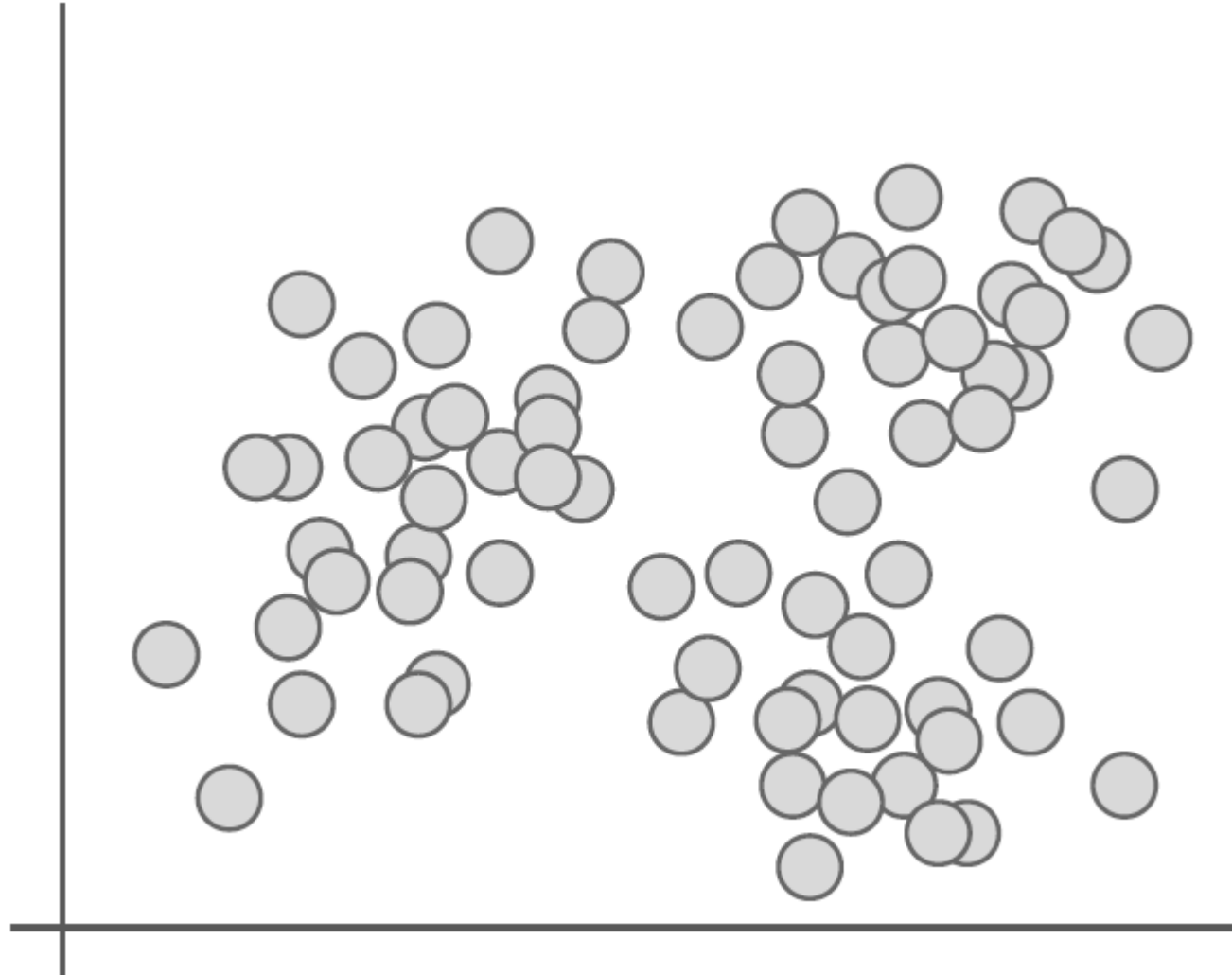
*os novos centroides são calculados como a média aritmética (centro geométrico) das amostras pertencentes a cada *cluster*.

- O algoritmo garantidamente chega a uma situação em que cada exemplo se encontra no *cluster* mais próximo, de modo que, a partir deste momento, os centroides não mudem mais.

Vejam os um exemplo

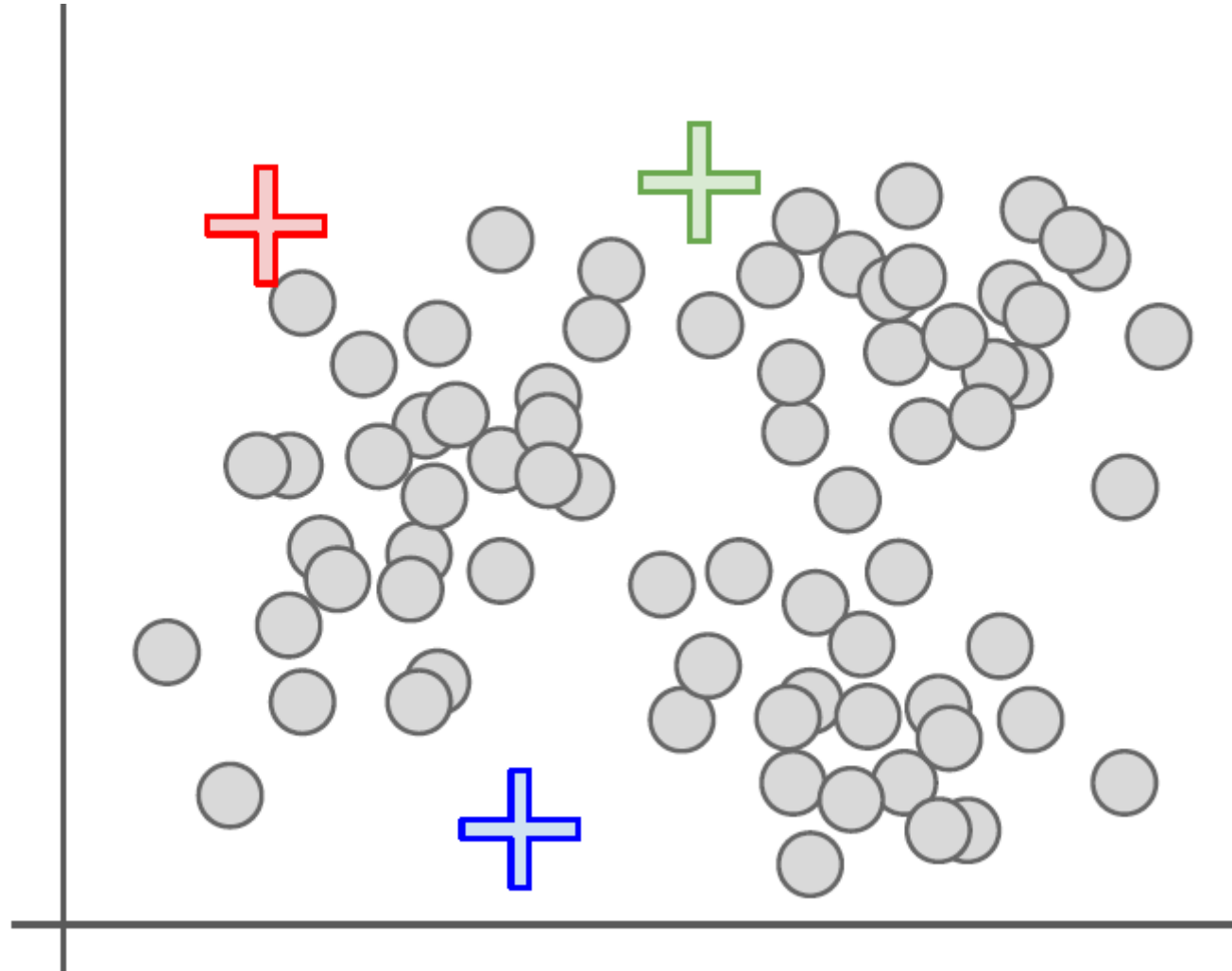
Funcionamento do *k-Means*

1. Define-se o número de clusters, $k = 3$



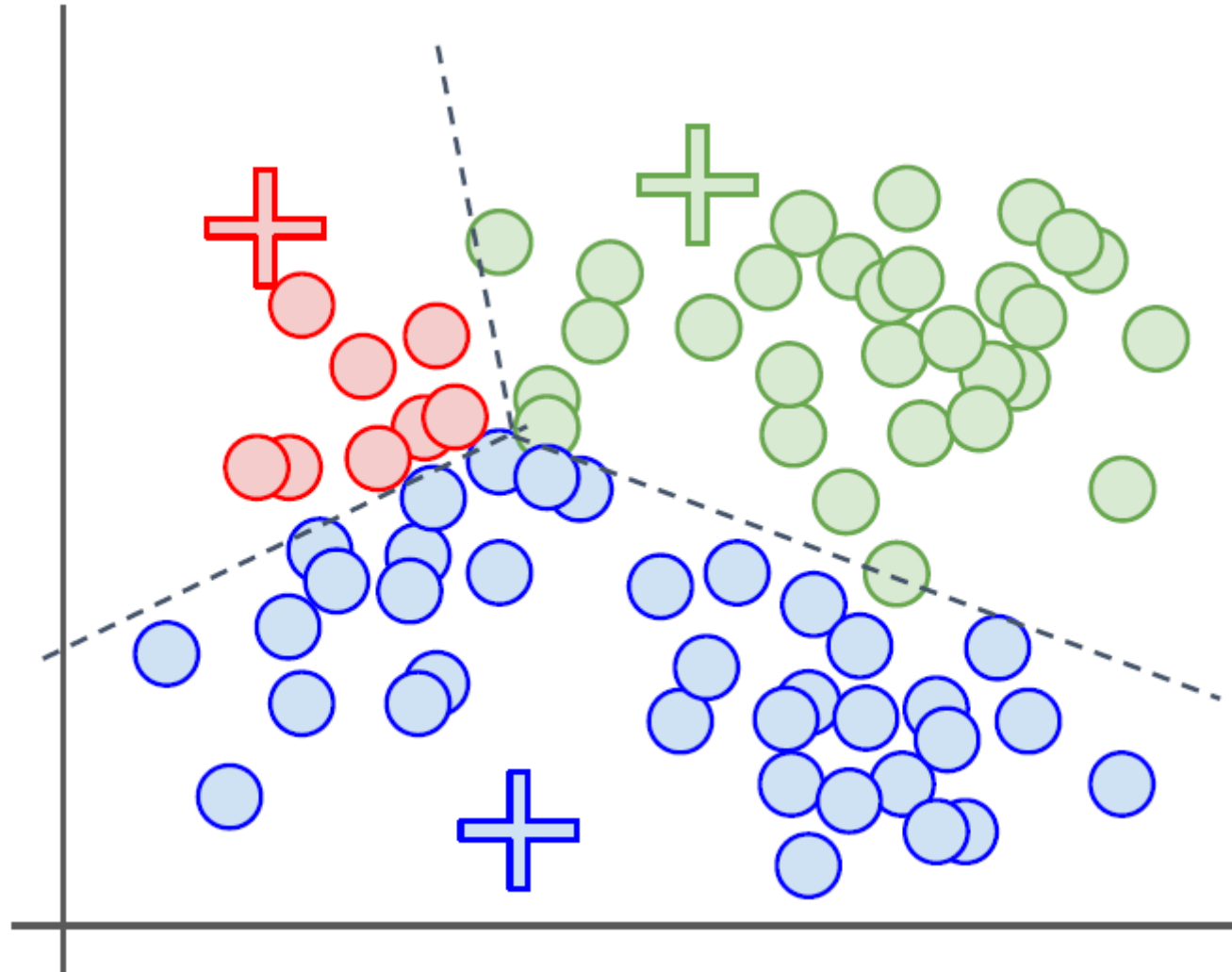
Funcionamento do *k-Means*

2. Escolhe-se os centroides para cada cluster de forma aleatória.



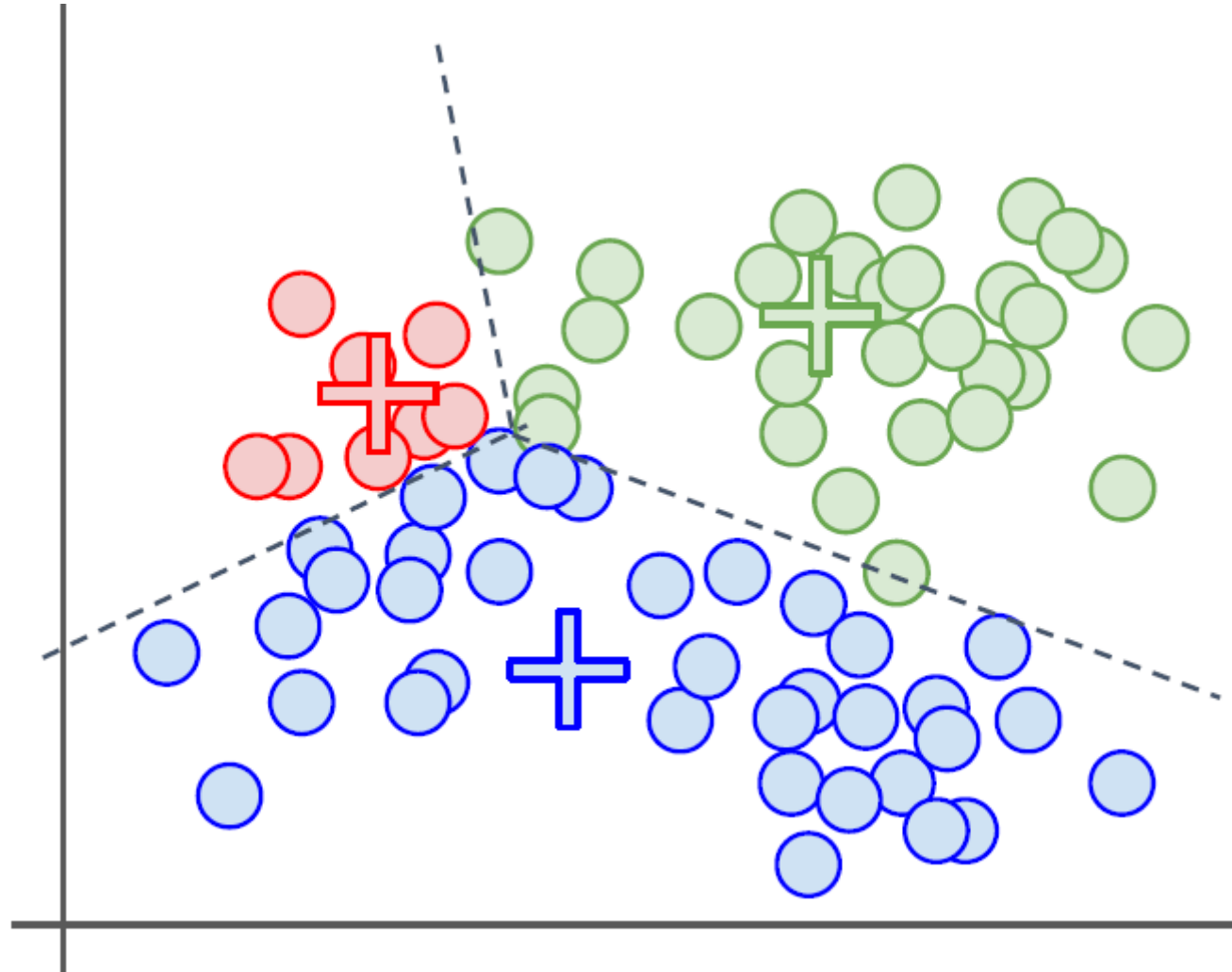
Funcionamento do *k-Means*

3. Atribui-se cada amostra ao centroide mais próximo com base na distância euclidiana.



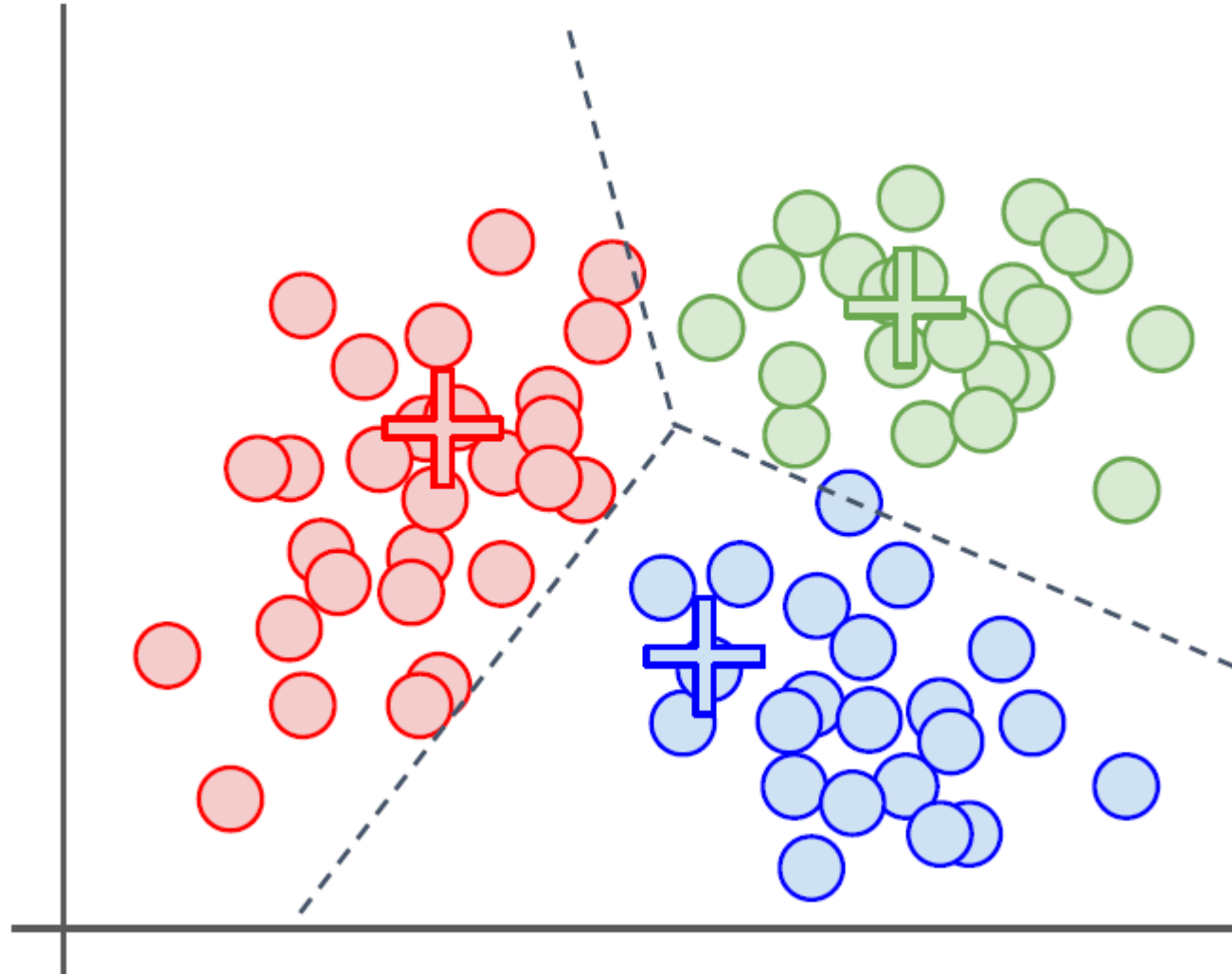
Funcionamento do *k-Means*

4. Recalcula-se o centroide dos clusters.



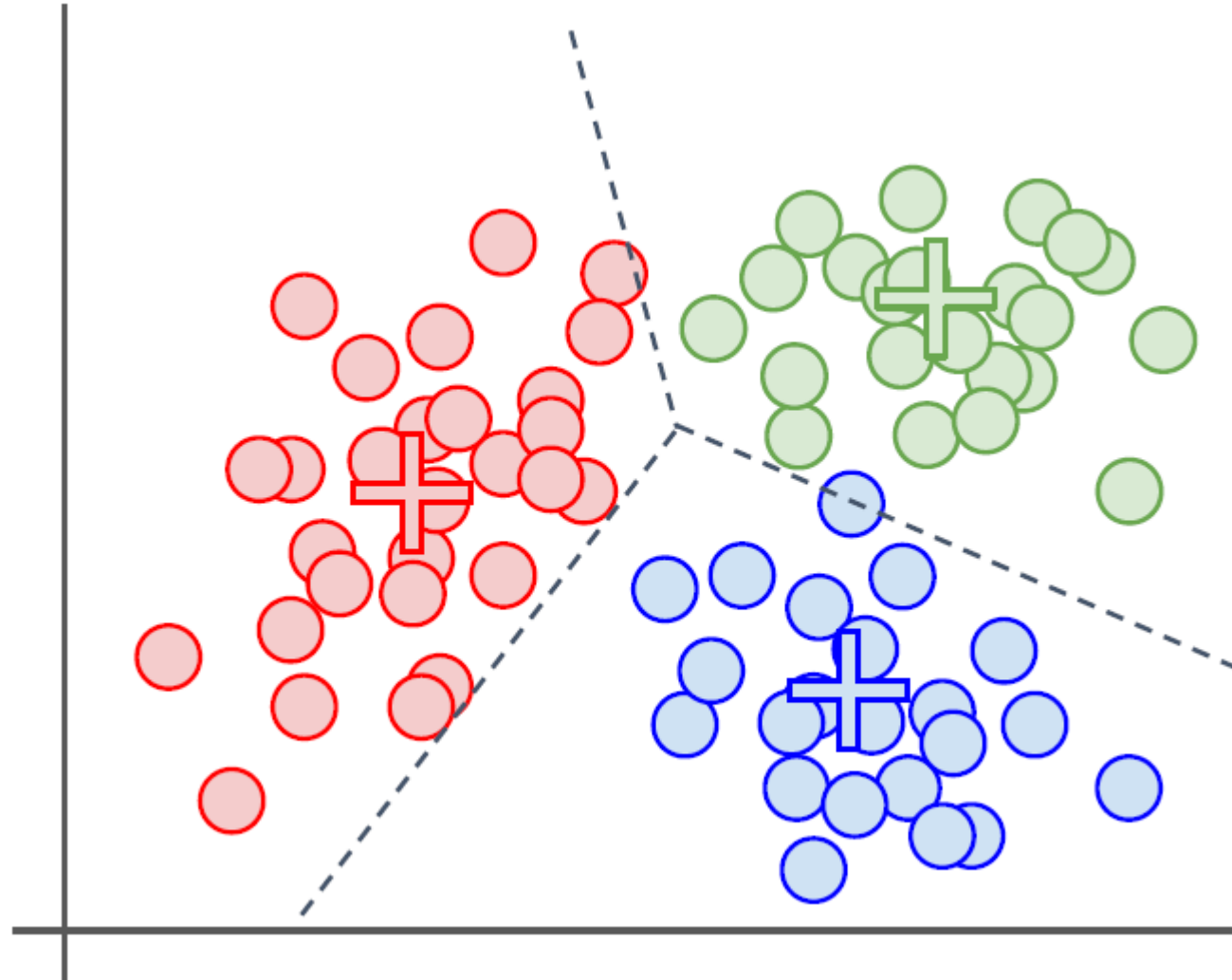
Funcionamento do *k-Means*

5. Repete-se os passos 3 e 4.



Funcionamento do *k-Means*

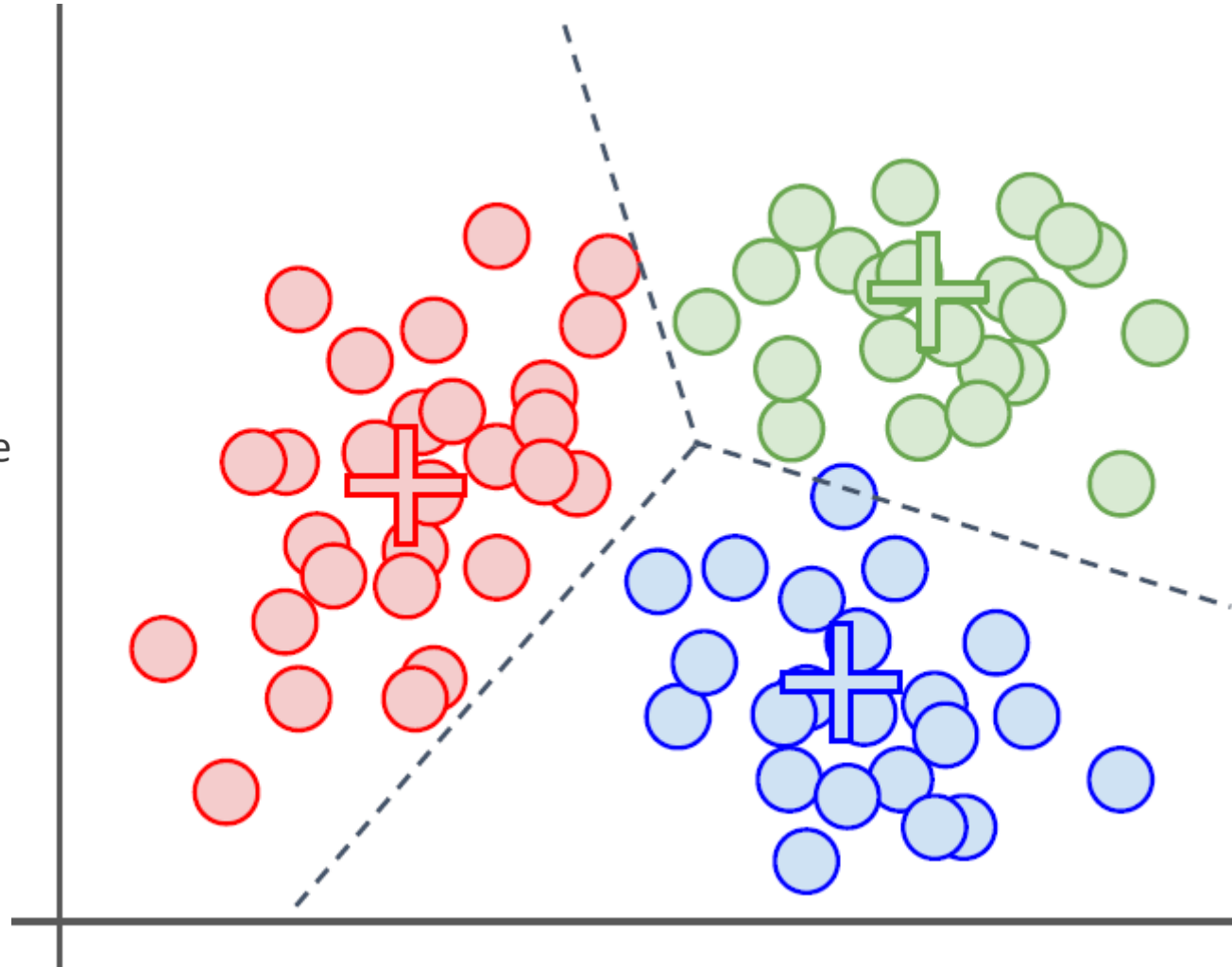
5. Repete-se os passos 3 e 4.



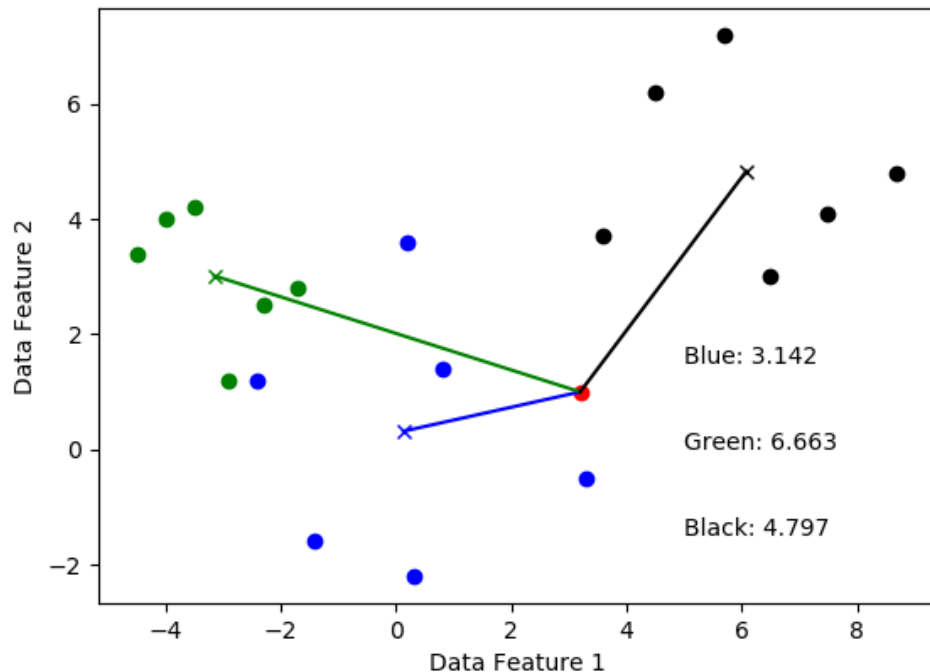
Funcionamento do *k-Means*

Repete-se os passos 3-4 até que um dos casos abaixo ocorra:

1. A soma das distâncias entre as amostras e o centroide correspondente seja minimizada
2. Não haja alteração nos centroides
3. O número máximo de iterações seja atingido

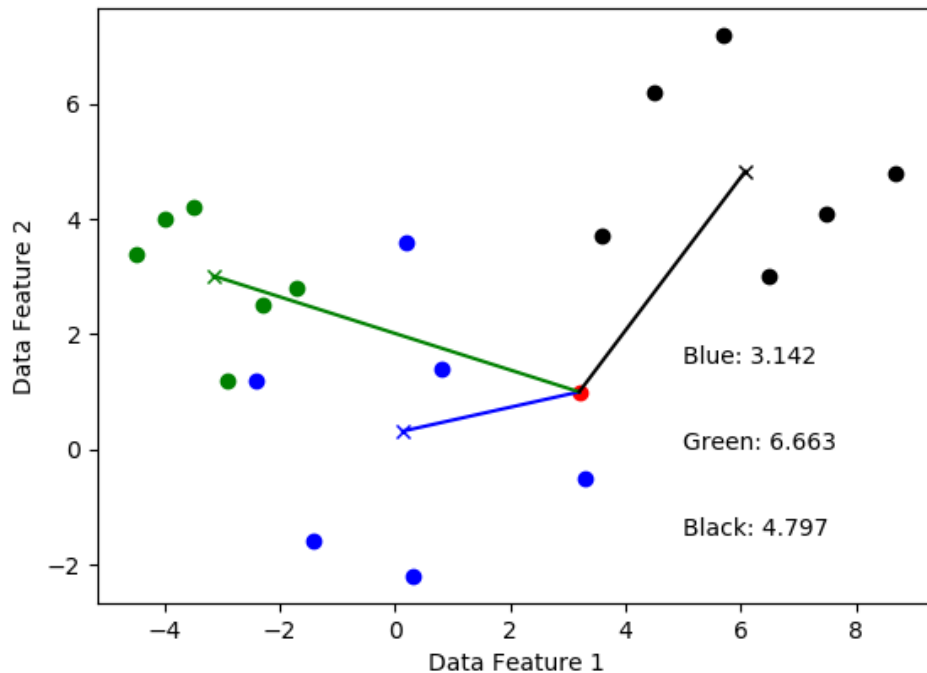


Como inicializar os centroides?



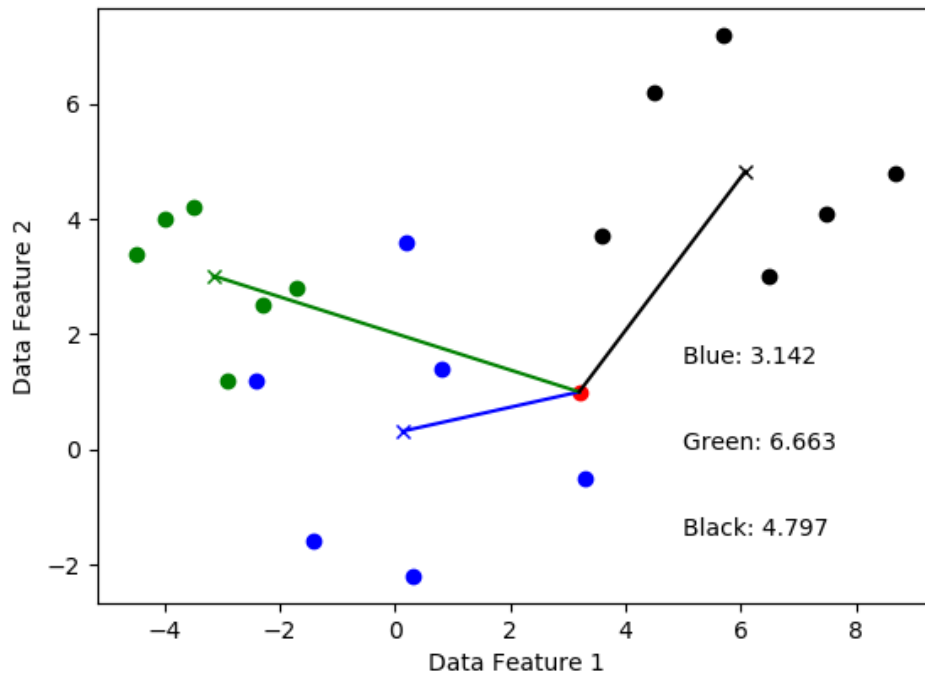
- O procedimento mais simples para inicializar os centroides é **escolher k exemplos de treinamento aleatórios** e os considerar como os centroides iniciais.
 - Pode escolher pontos muito próximos
 - Rodar várias vezes e escolher o melhor resultado
- Os **clusters iniciais** são então criados associando cada um dos exemplos ao seu **centroide mais próximo**.
- Cada centroide define uma região e as amostras são distribuídas com base nisso.

Como inicializar os centroides?



- Se os centroides iniciais mudam:
 - muda completamente a divisão inicial
 - muda quais amostras vão para qual *cluster* no início
- O número de **transferências** de um exemplo **de um *cluster* para outro depende dos centroides iniciais**.
 - Todo o processo depende do ponto de partida.
- Se os **centroides iniciais** já forem **perfeitos**, **nenhum exemplo precisa ser atribuído a outro *cluster*** e o algoritmo é encerrado.

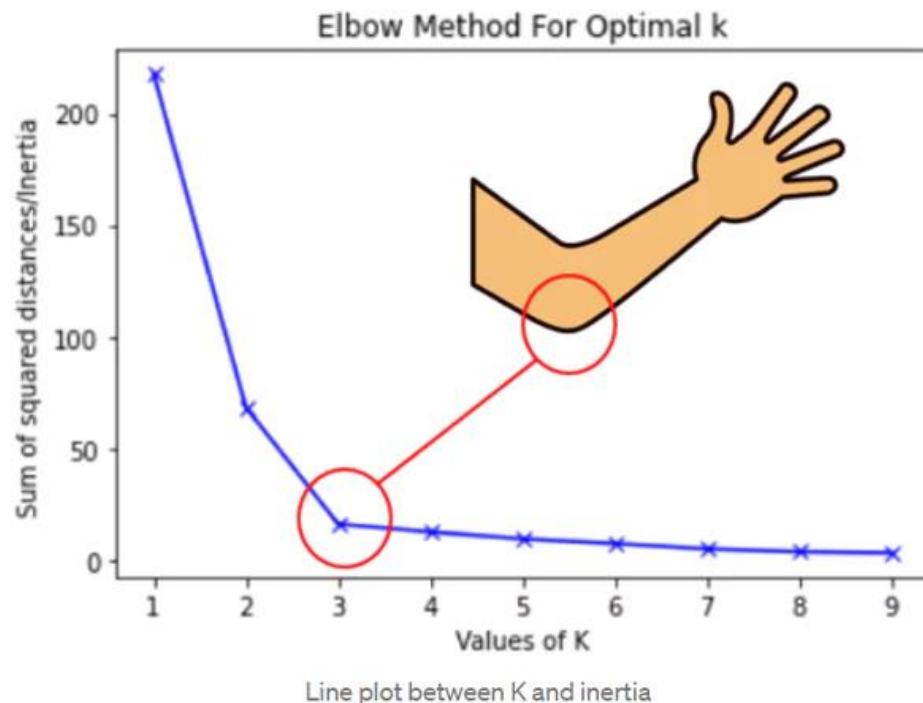
Como inicializar os centroides?



- Portanto, a **inicialização é importante** no sentido de que **um ponto de partida melhor** garante que a **solução seja encontrada mais rápido**.
- Ou seja, o **tempo de convergência** do algoritmo **depende da inicialização**.
- O algoritmo é **altamente sensível à escolha inicial dos centroides**.

Como definimos o número de
clusters?

Método do cotovelo



- O método busca o ponto onde a **melhoria do aumento de *clusters* começa a diminuir significativamente.**
- Ou seja, aumentar k sempre melhora o ajuste, mas chega um ponto em que o ganho é **marginal.**
- Esse ponto é o “**cotovelo**” da curva.
- Que métrica usamos para plotar a curva?

Soma dos quadrados intra-*clusters*

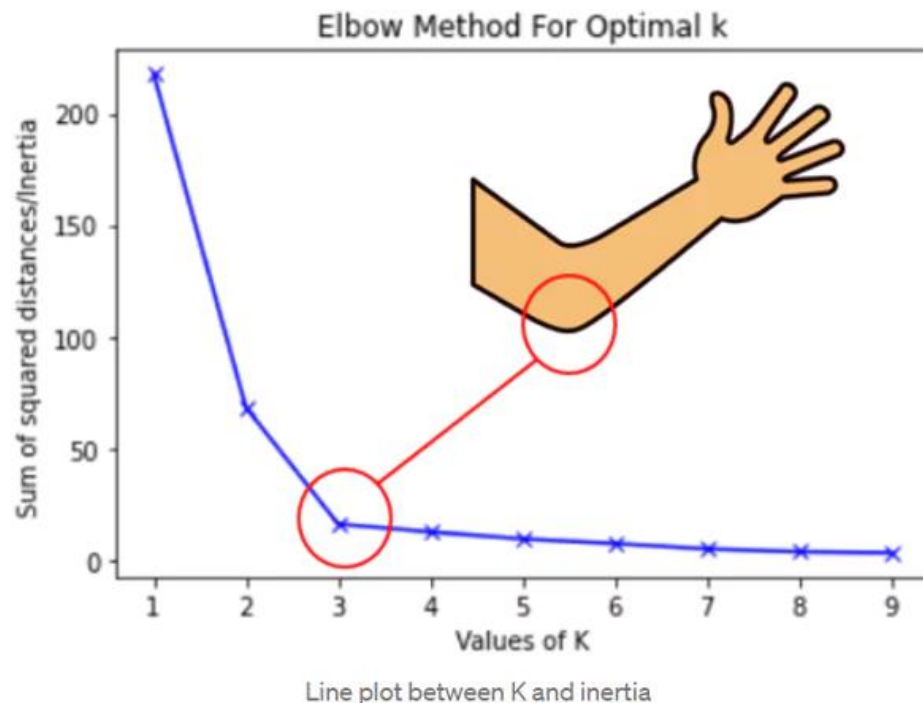
- Usa-se a soma dos quadrados intra-*clusters*, também conhecida como inércia

$$I(k) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

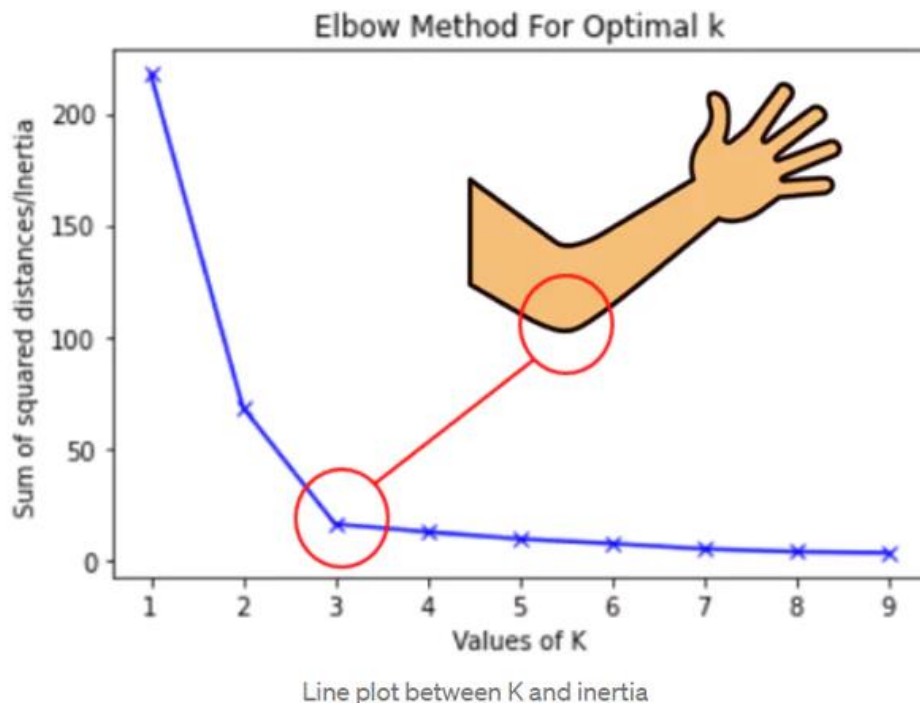
onde μ_i é o centroide do i -ésimo *cluster*, C_i é conjunto de amostras no i -ésimo *cluster* e $\|x - \mu_i\|^2$ é a distância até μ_i .

- Ela mede o quão “compactos” são os *clusters*.

- quanto menor → melhor

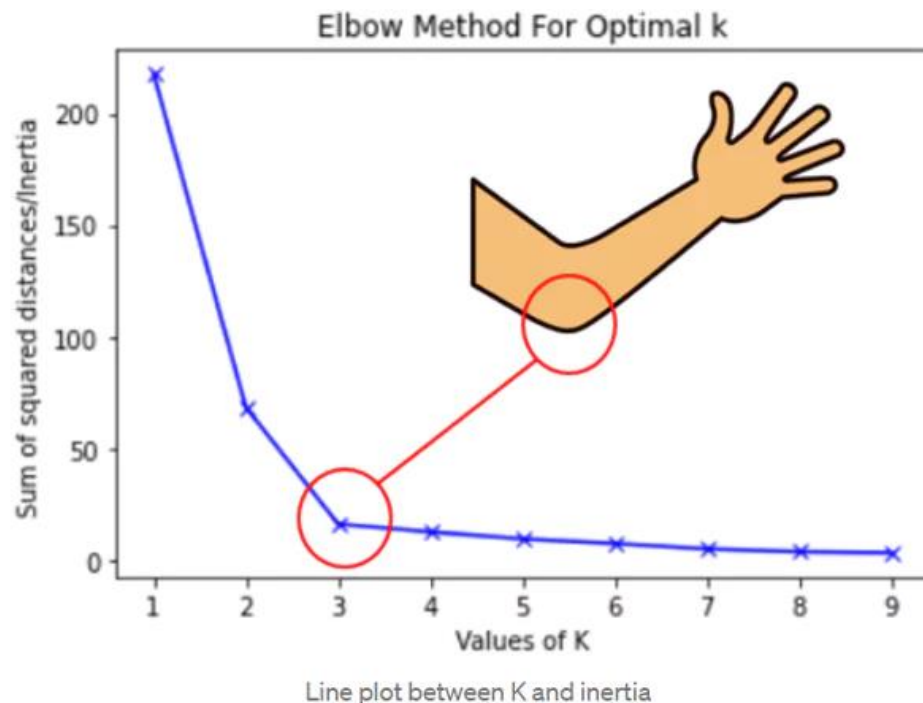


Soma dos quadrados intra-*clusters*



- Plota-se o valor da inércia para um intervalo de valores de k .
- A curva resultante quantifica o quão próximos os pontos de um cluster estão do seu centroide.
- Quanto menor a inércia, mais coesos são os *clusters*:
 - os pontos dentro de um *cluster* estão fortemente agrupados, próximos entre si e do centroide.

Soma dos quadrados intra-*clusters*



- Assim, o valor ideal de k é encontrando localizando-se o cotovelo na curva.
- Quando o cotovelo não é visível (transição suave), uma técnica comum é traçar uma reta conectando os pontos extremos da curva e identificar o ponto mais distante dessa reta como o "cotovelo" ideal.

Outros algoritmos de *clustering*

- DBSCAN: clusterização baseada em densidade.
 - *Clusters* são regiões **densas** separadas por regiões vazias.
- HDBSCAN: Extensão do DBSCAN que constrói uma **hierarquia de densidades**.
- *Gaussian mixtures*: Assume que os dados vêm de uma mistura de **distribuições gaussianas**.
- Documentação da biblioteca *scikit-learn* sobre clusterização:
 - <https://scikit-learn.org/stable/modules/clustering.html>

Exemplo

- [Exemplo: kmeans.ipynb](#)

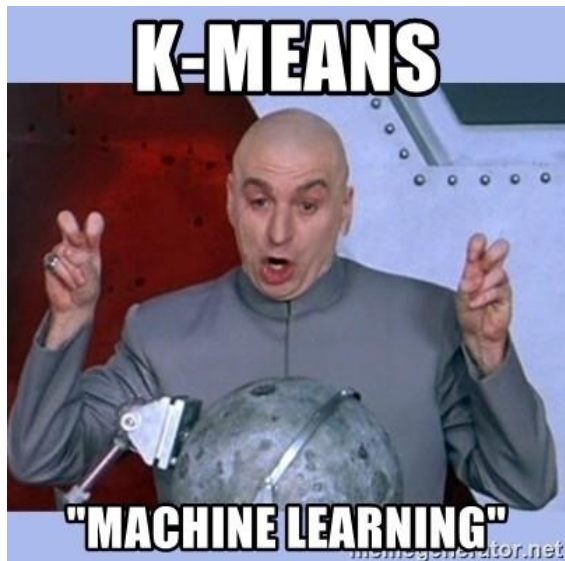


Lista de exercícios

Lista #13 – kmeans

Perguntas?

Obrigado!



k-means be like:



K-MEANS CLUSTER

